



Networking, Cloud Basics & Cybersecurity Fundamentals

Session 3 — Enterprise Engineering Foundations · 3 Hours

Networking

TCP/IP, DNS, routing, load balancers

Cloud & AWS

Architecture, VPC, core services

Security

Zero Trust, Defense in Depth, security zones

Applied

Mortgage platform deployment topology

Session Overview

Modern enterprise applications depend on both application code *and* network infrastructure. Understanding the full journey is essential.

01

Request Lifecycle

Browser → backend: TCP/IP, DNS, ports, routing, NAT, HTTP/HTTPS, TLS

02

Cloud & AWS

Architecture fundamentals, VPC, subnets, and core AWS services

03

Security Architecture

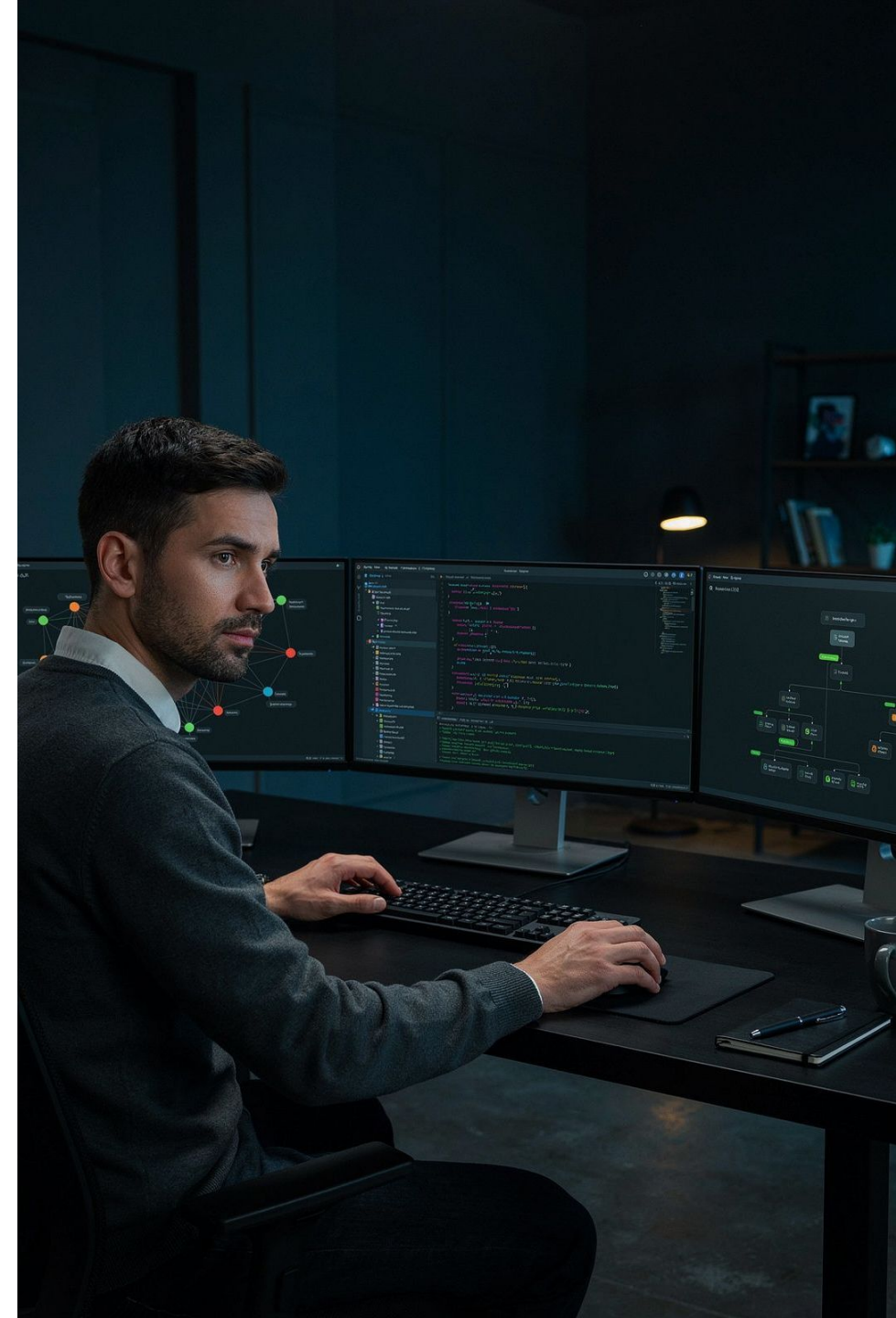
Zero Trust, Defense in Depth, zones, WAF, IDS/IPS, VPN, DMZ

04

Mortgage Platform

Secure document exchange, trust boundaries, deployment topology

📄 **Main Goal:** Understand how networking, cloud, and cybersecurity work together in a real enterprise mortgage application.



Our Mortgage Application Scenario

One real-world scenario used throughout the session: **a borrower uploads an income document to the Mortgage Portal.**



Data in Motion

- Borrower & loan information
- Income documents & financial records
- Mortgage servicing data
- Property information

□ Every arrow = a network communication and a potential security boundary.

Networking Fundamentals for Distributed Systems

Networking allows applications on different systems to communicate — browser to API, API to database, microservice to microservice.

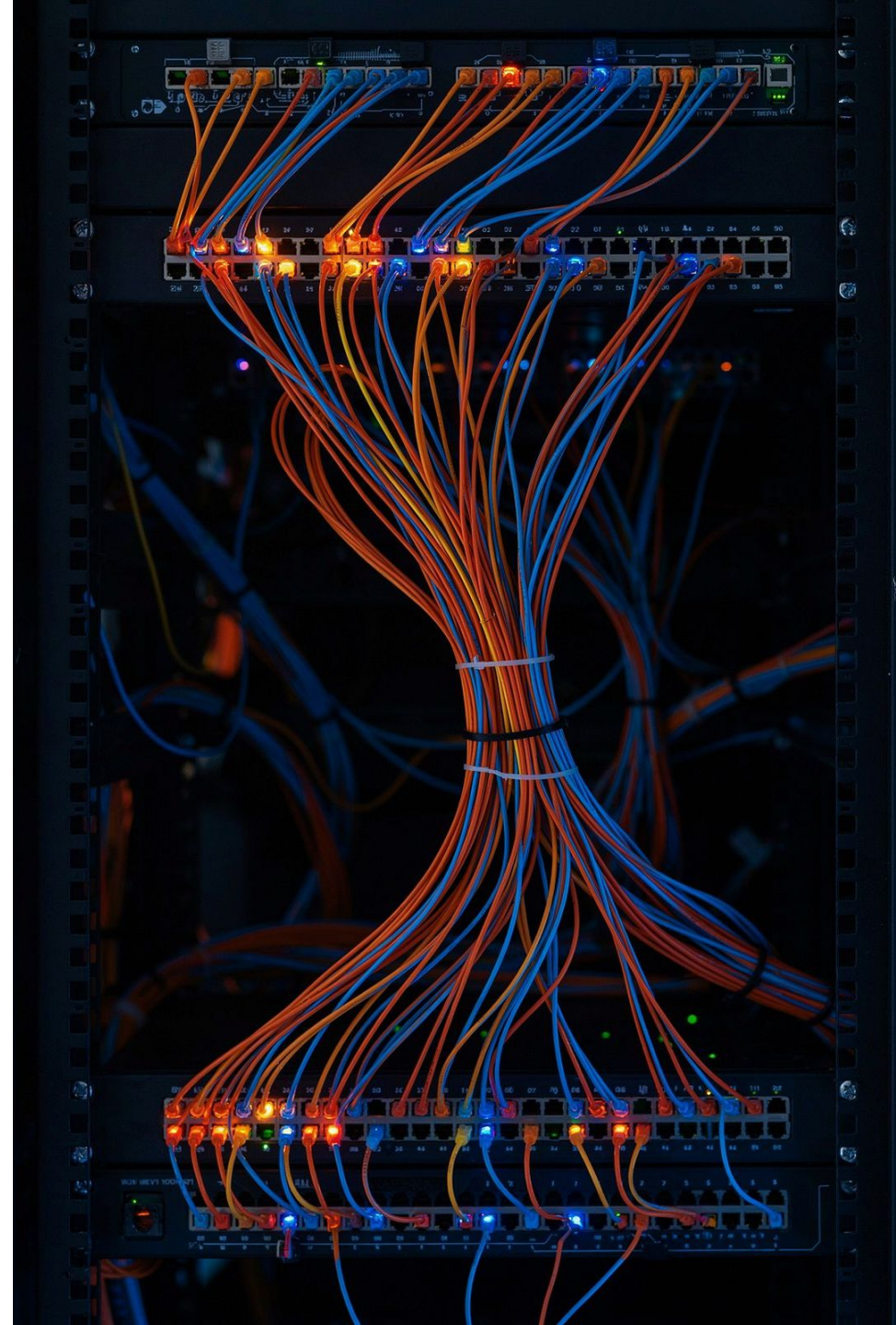
Every Communication Requires

- Source & Destination
- IP Address & Port
- Protocol & Route
- Security Rule

Mortgage API → PostgreSQL

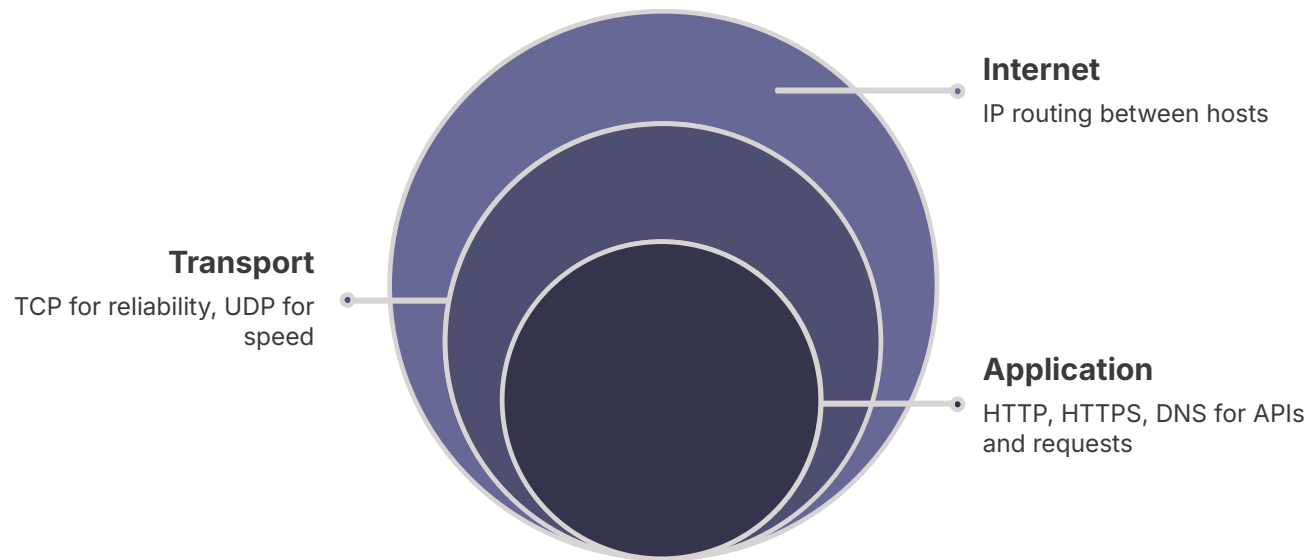
- IP: 10.0.3.20
- Port: 5432
- Protocol: TCP

 Many application failures are actually **connectivity problems** — not code defects.



TCP/IP Model

The foundation of all modern network communication — four layers, each with a specific role.



Mortgage API Example

- **HTTP** — defines the API request shape
- **TCP** — guarantees reliable delivery
- **IP** — routes packets to the right host
- **Network** — physically transports the data

Each layer adds its own header. Each layer solves one problem.

IP Addresses and Ports

The Mental Model

IP Address

The building — identifies the host on the network

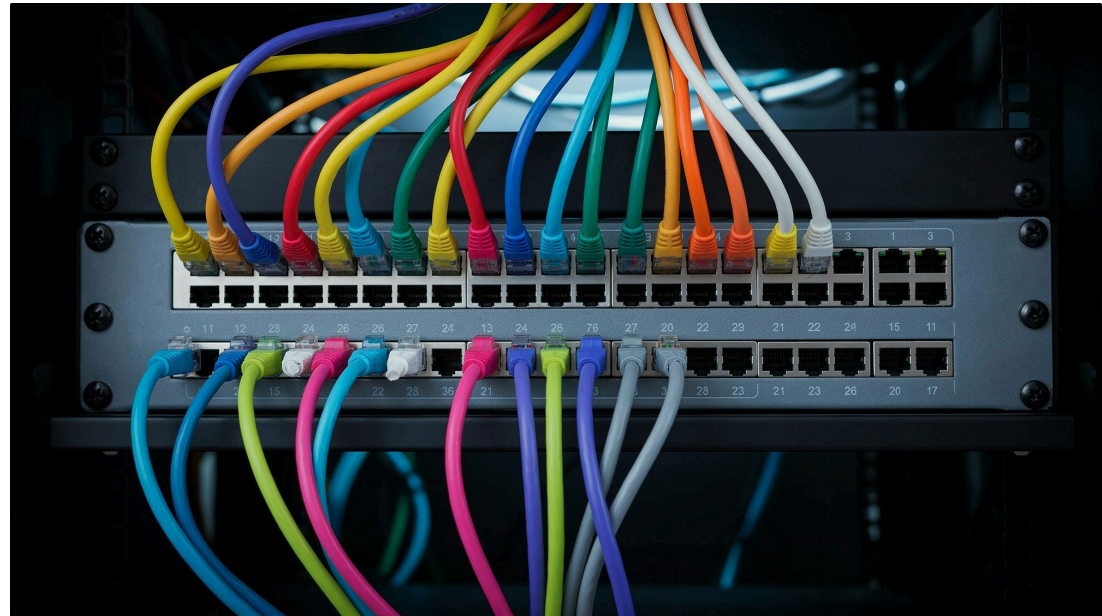
Port

The room — identifies which service to reach

Common Ports

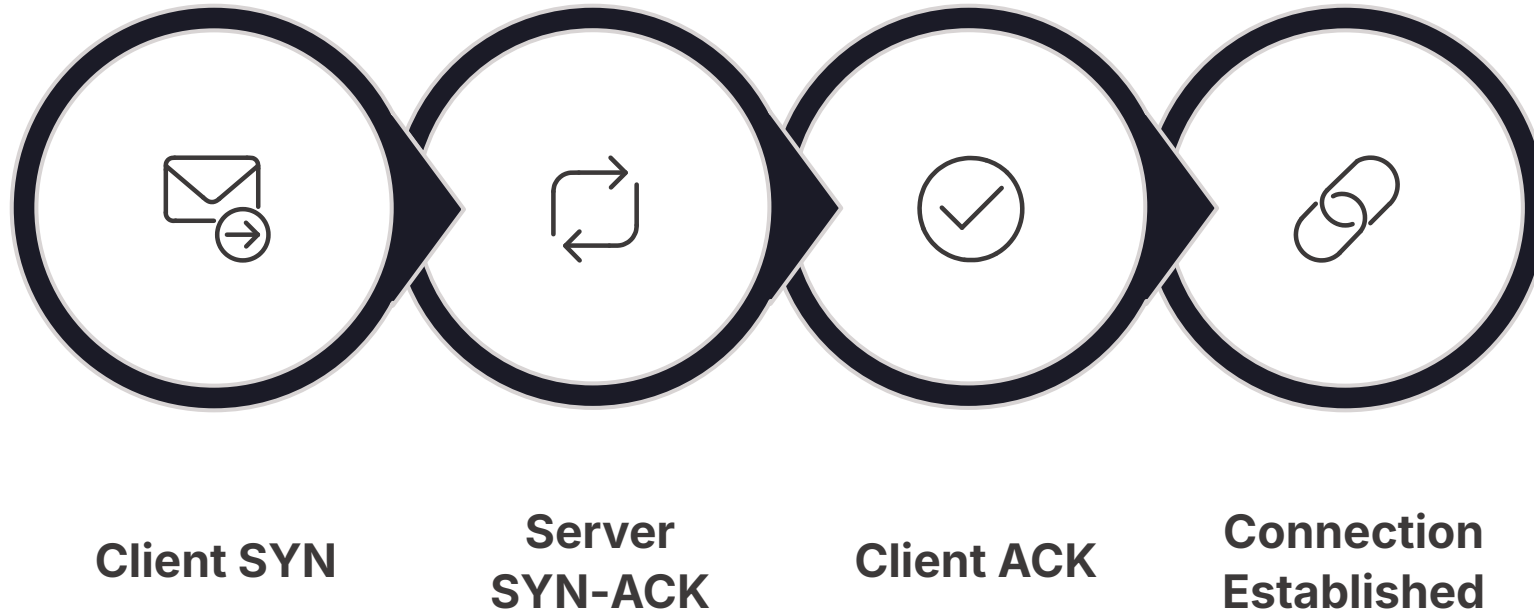
- HTTP — 80
- HTTPS — 443
- SSH — 22
- DNS — 53
- PostgreSQL — 5432
- MySQL — 3306

 **Security Principle:** Expose only the ports that are actually required.



TCP and the Three-Way Handshake

Before any application data flows, TCP establishes a reliable connection.



Connection Timed Out

TCP could not establish a connection — likely a **network-level** problem: firewall, routing, or wrong IP.

401 Unauthorized

The API *was* reached. This is an **application-level** problem — not a network failure.

DNS: Domain Name System



Name → Address

api.mortgage.example.com → 203.0.113.25

Request Lifecycle

- Browser triggers DNS resolution
- IP address returned
- TCP connection established
- TLS handshake, then HTTPS request

❌ **Could not resolve host** — if DNS fails, the request never reaches the application.

HTTP, HTTPS & Status Codes

HTTP Methods

GET

Retrieve

POST

Create

PUT

Replace

PATCH

Update

DELETE

Remove

Key Status Codes

- **200** OK · **201** Created
- **400** Bad Request
- **401** Unauthorized · **403** Forbidden
- **404** Not Found
- **500** Internal Server Error
- **502** Bad Gateway · **503** Unavailable

📌 HTTPS = HTTP + TLS. All sensitive mortgage data must travel over encrypted connections.

TLS and Secure Communication

Transport Layer Security provides three guarantees



Confidentiality

Data cannot be read in transit



Integrity

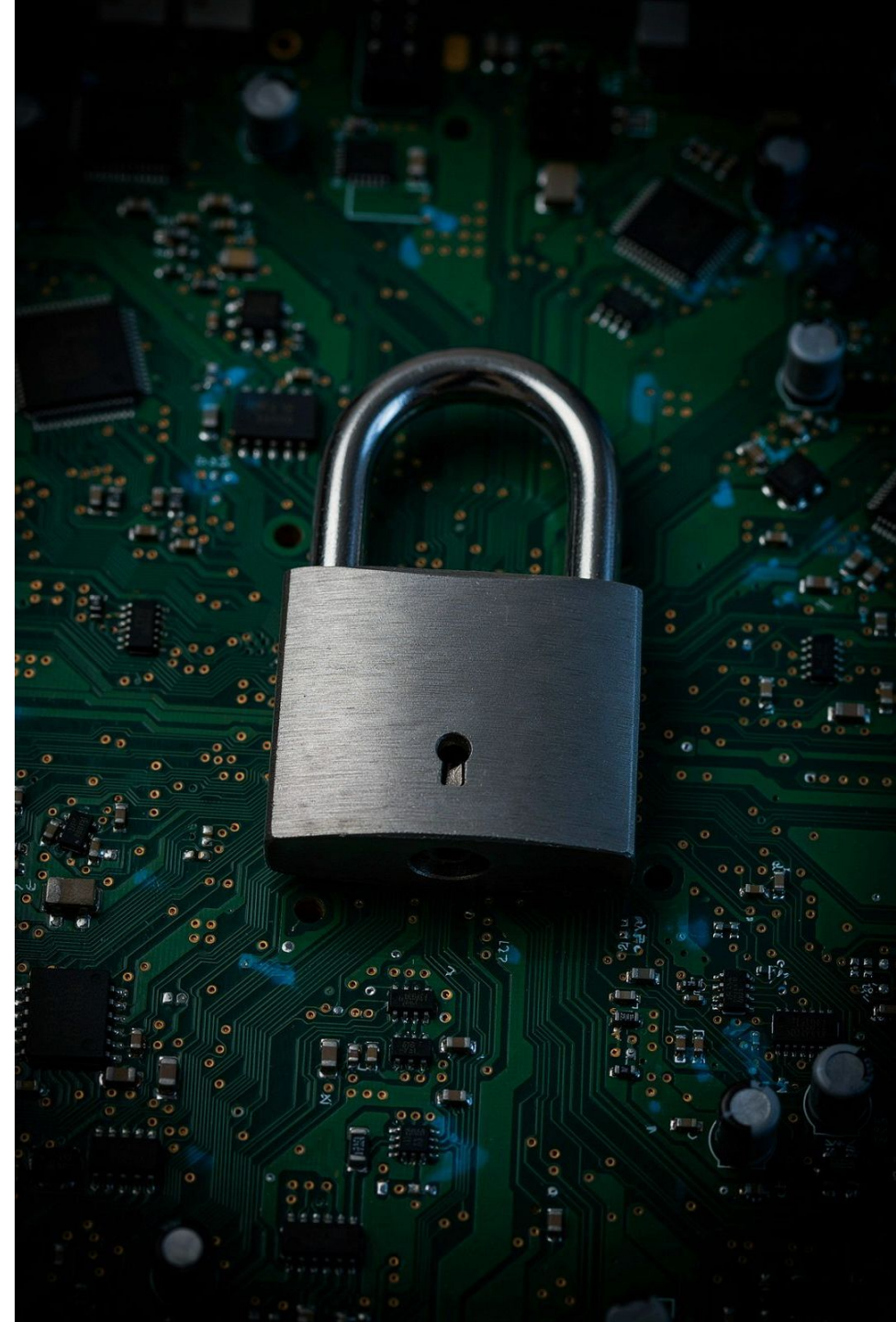
Tampering is detectable



Authentication

Certificates verify server identity

Borrower PII, loan details, and financial documents must **always** be transmitted over TLS-encrypted connections.



Routing and NAT

Routing

Route tables determine how traffic moves between networks.

Mortgage API

10.0.2.10

Database

10.0.3.20

NAT — Network Address Translation

Private servers make outbound internet connections *without* being publicly reachable.

Private AI Service

NAT Gateway

Approved External Service

The application connects outbound while remaining private.

Load Balancers



Without Load Balancing

All users → Single API server

With Load Balancing

Users → Load Balancer → API-1 · API-2 · API-3

Benefits

- Scalability & high availability
- Health checks & automatic failover
- Distributes traffic across instances

📄 During peak servicing or refinancing activity, traffic is spread across multiple Mortgage API instances.

Cloud Architecture Basics

Cloud is not just hosting. It is designing systems for **scalability, security, resilience, and automation.**



Compute & Containers

Virtual machines, containers, serverless functions



Storage & Databases

Managed relational DBs, object storage, caching



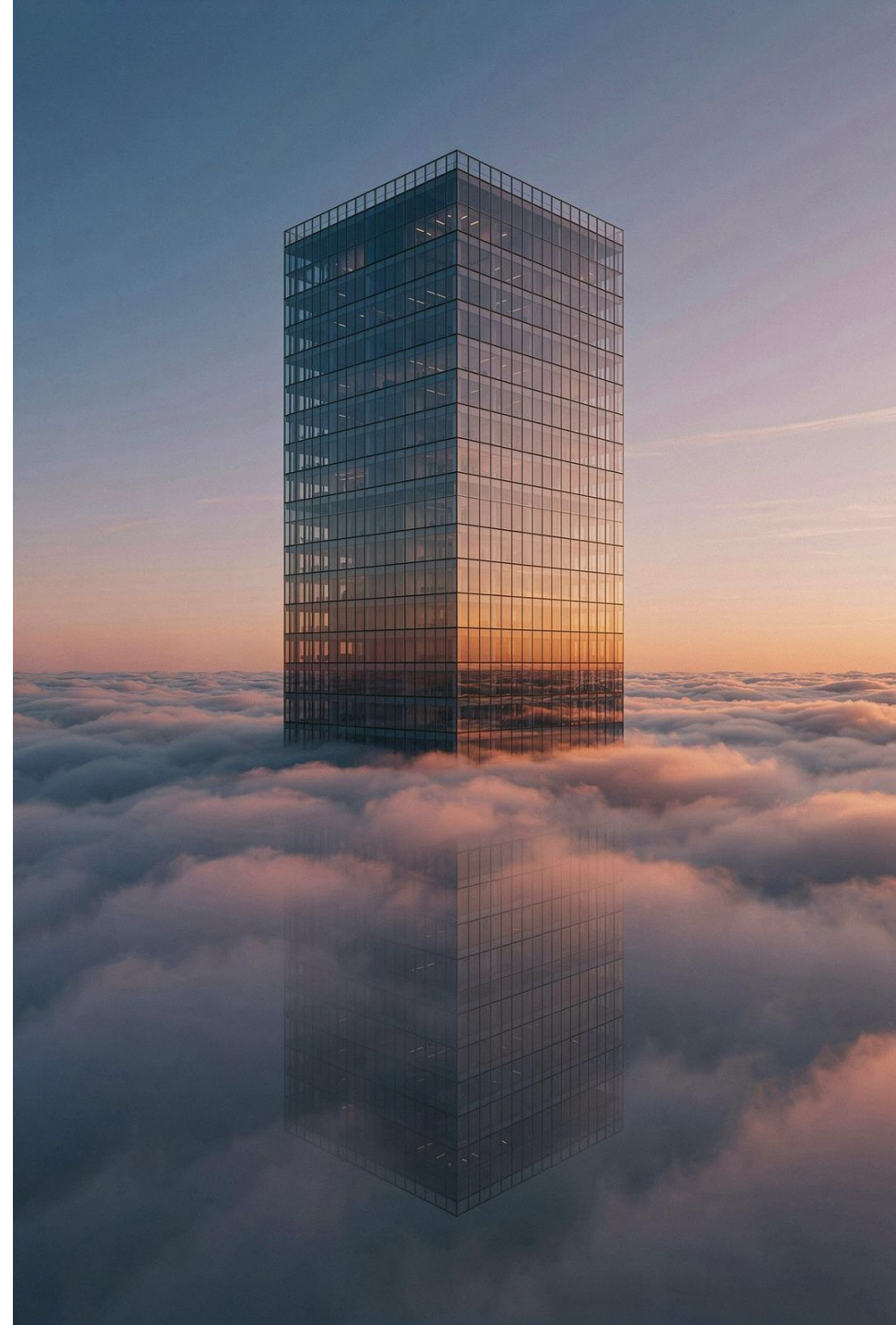
Identity & Security

IAM, key management, secrets, audit



Networking & Monitoring

VPC, DNS, load balancers, observability



AWS Ecosystem Overview

Core AWS services for the mortgage platform — covered in depth in later sessions.

Networking

- VPC
- Route 53
- NAT Gateway
- Security Groups

Compute

- EC2
- ECS / EKS
- Lambda

Storage & DB

- S3
- RDS

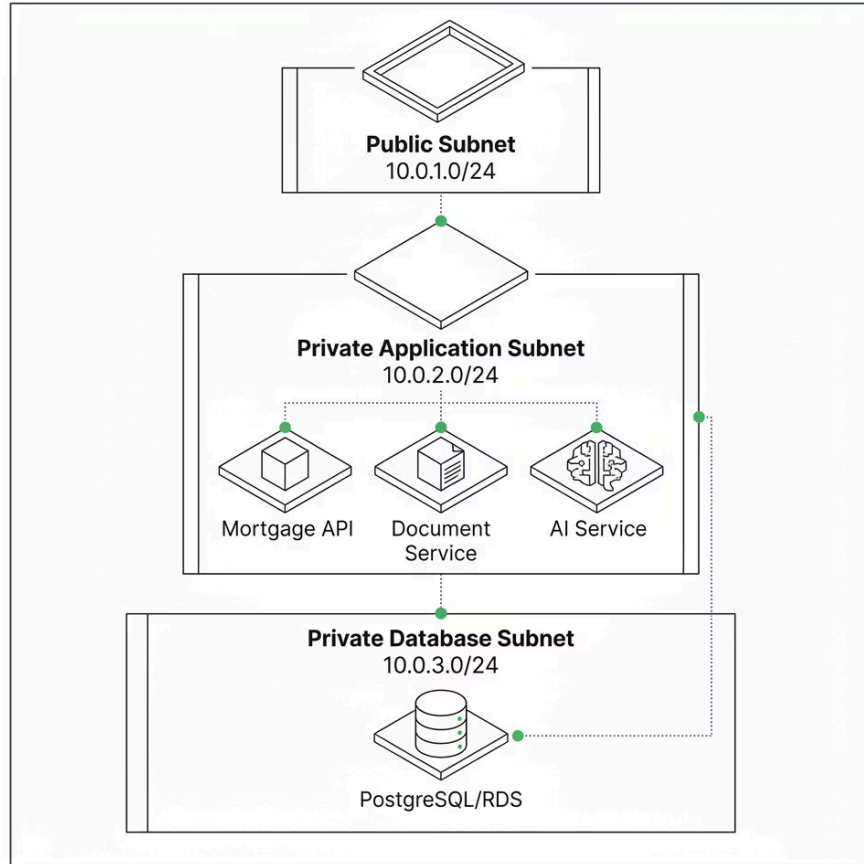
Security

- IAM
- KMS
- Secrets Manager
- AWS WAF

Monitoring

- CloudWatch

AWS VPC and Subnets



VPC: 10.0.0.0/16

Public Subnet · /24

Internet-facing load balancer only

App Subnet · /24

Mortgage API, Document Service, AI Service

Data Subnet · /24

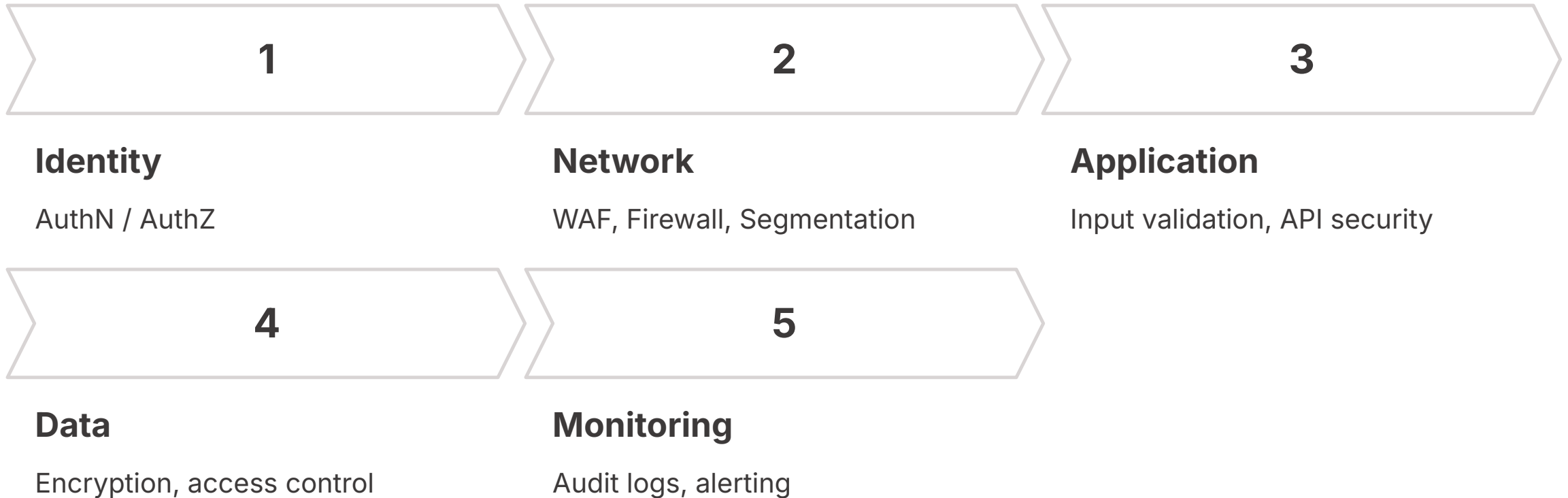
PostgreSQL / RDS — no public access



Principle: Keep backend and data services private whenever possible.

Enterprise Security Architecture

Security is not one product — it is a system of coordinated, layered controls.



ZERO TRUST

Zero Trust Principles

✗ Old Model

Inside network = Trusted

✓ Zero Trust

Never trust based on location alone

Verify Explicitly

Validate identity and context every time

Least Privilege

Grant only what is required

Assume Breach

Design as if an attacker is already inside

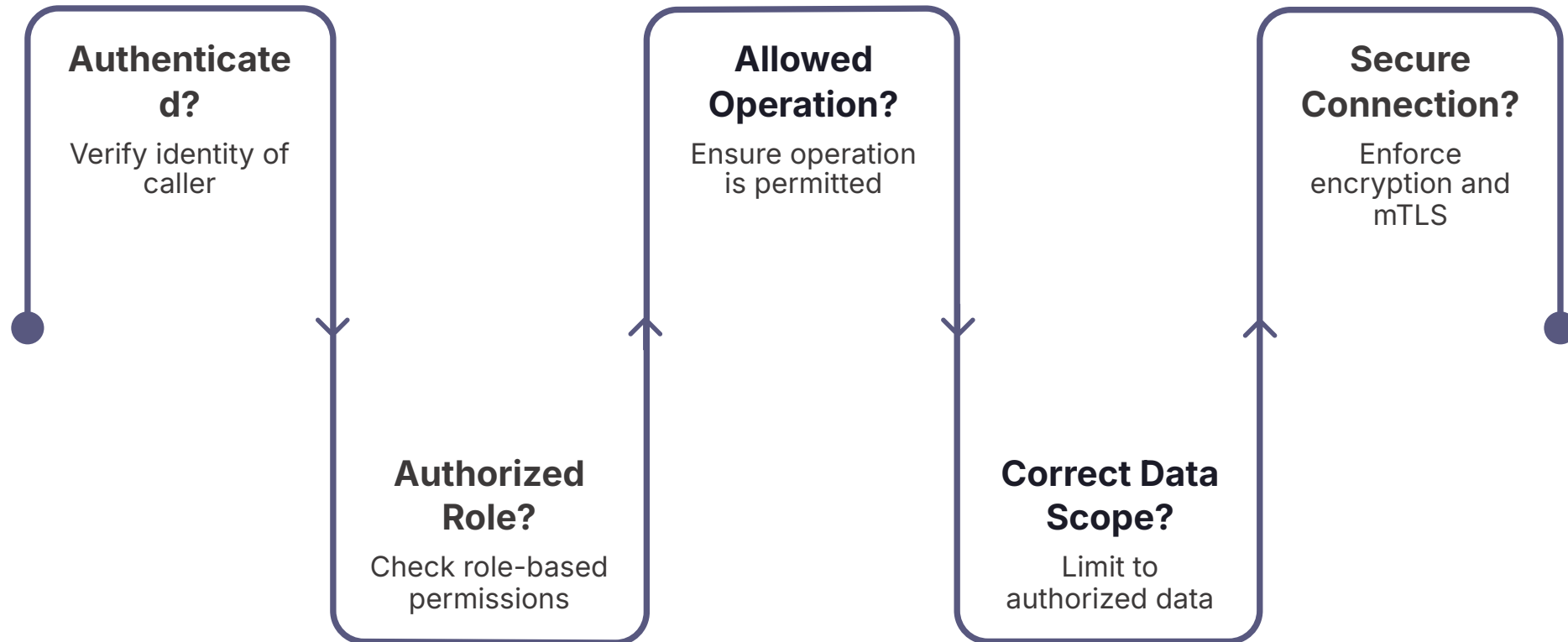
Continuously Validate

Trust is never permanent



Zero Trust in the Mortgage Platform

Employee calls GET /api/admin/borrowers — do not simply ask "Is this person on the VPN?"



❏ Zero Trust also applies **between services**: the Mortgage API calling the AI Service must still carry service identity, authorization, and audit logging.

Defense in Depth

Multiple security layers — if one fails, others still protect the application.

1

Monitoring

Detects suspicious activity

2

Encryption

Data protected at rest and in transit

3

DB Permissions

Least-privilege database accounts

4

Input Validation

Rejects malicious payloads at the app layer

5

Authentication

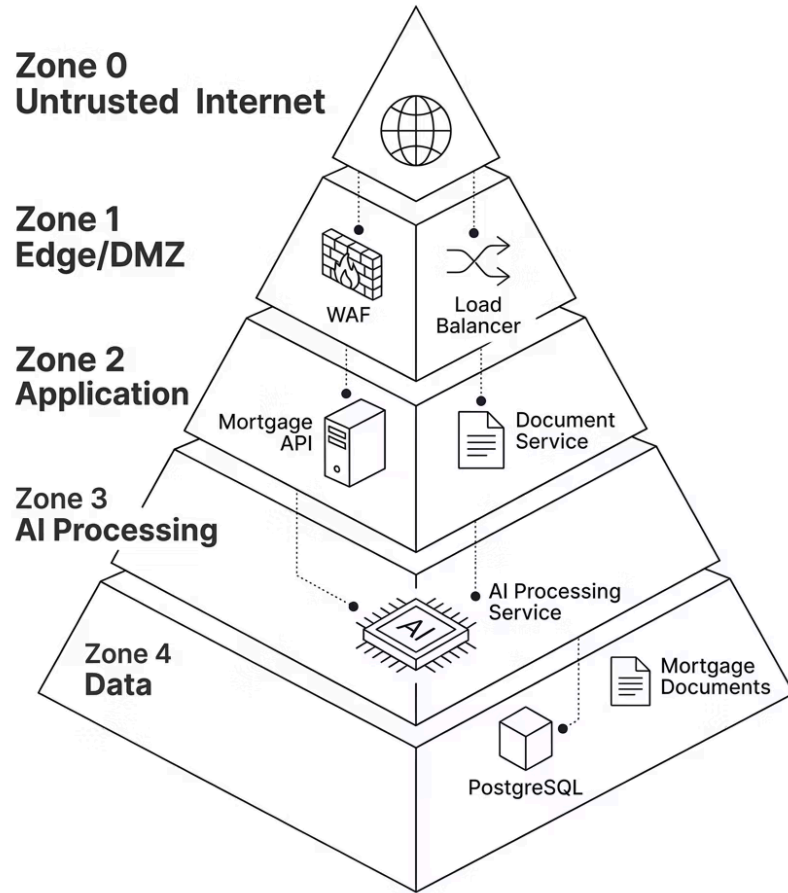
Verifies every identity

6

WAF / Firewall

First line of network defense

Security Zones and Network Segmentation



Why Segmentation Matters

Systems cannot communicate freely across zones — **all traffic must match an approved rule.**

- Limits blast radius of a breach
- Prevents lateral movement
- Enforces least-privilege networking
- Provides clear audit boundaries

❏ Separate systems by trust level and data sensitivity — not just by application team.

Ingress, Egress and Network Rules

Ingress — Traffic Entering

Internet → Load Balancer

Egress — Traffic Leaving

AI Service → Approved External Endpoint

Mortgage Rule Examples

Source	Destination	Port	Action
Internet	ALB	443	ALLOW
Internet	PostgreSQL	5432	DENY
ALB	Mortgage API	—	ALLOW
Mortgage API	PostgreSQL	5432	ALLOW
Internet	AI Service	—	DENY

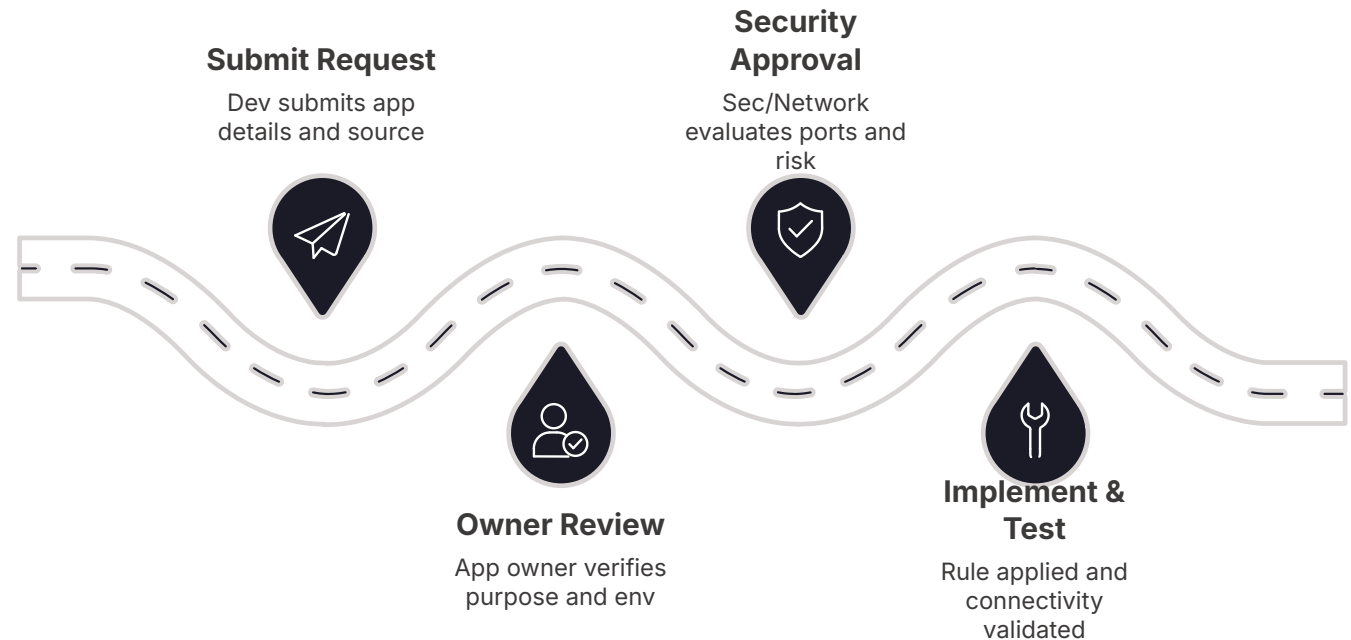
Firewall and Enterprise Connectivity Approvals

What a Firewall Evaluates

- Source & Destination
- Port & Protocol
- Direction & Action

Approval Request Typically Includes

- Application name & owner
- Source IP / subnet
- Destination, port, protocol
- Environment & business justification



IDS, IPS and WAF



IDS

Intrusion Detection System

Monitors traffic and **alerts** on suspicious patterns — does not block



IPS

Intrusion Prevention System

Monitors and can **block** malicious traffic inline



WAF

Web Application Firewall

Protects HTTP/HTTPS — detects SQL injection, malicious bots, common web attacks, abusive API requests



Easy memory: IDS = Detect · IPS = Prevent · WAF = Protect Web/Application Traffic

Proxy, VPN and DMZ

Proxy

Application → Corporate Proxy → Internet

- Outbound traffic control & logging
- Content filtering & policy enforcement

VPN

Developer → Encrypted Tunnel → Corporate Network

- Secure remote connectivity
- VPN ≠ authorization — additional controls still required

DMZ

Internet → Firewall → DMZ → Firewall → Internal Systems

- Separates public-facing resources from sensitive internal systems

Security Monitoring Basics

Security architecture must provide **visibility** — unknown events cannot be defended against.

Monitor For

- Failed logins & 401/403 spikes
- WAF blocks & firewall denies
- Unusual or unexpected outbound traffic
- Database connectivity failures
- TLS certificate errors & high request rates

Mortgage Example

Normal

20 failed logins / hour

Anomaly

15,000 failed logins in 10 minutes → Brute force / credential stuffing



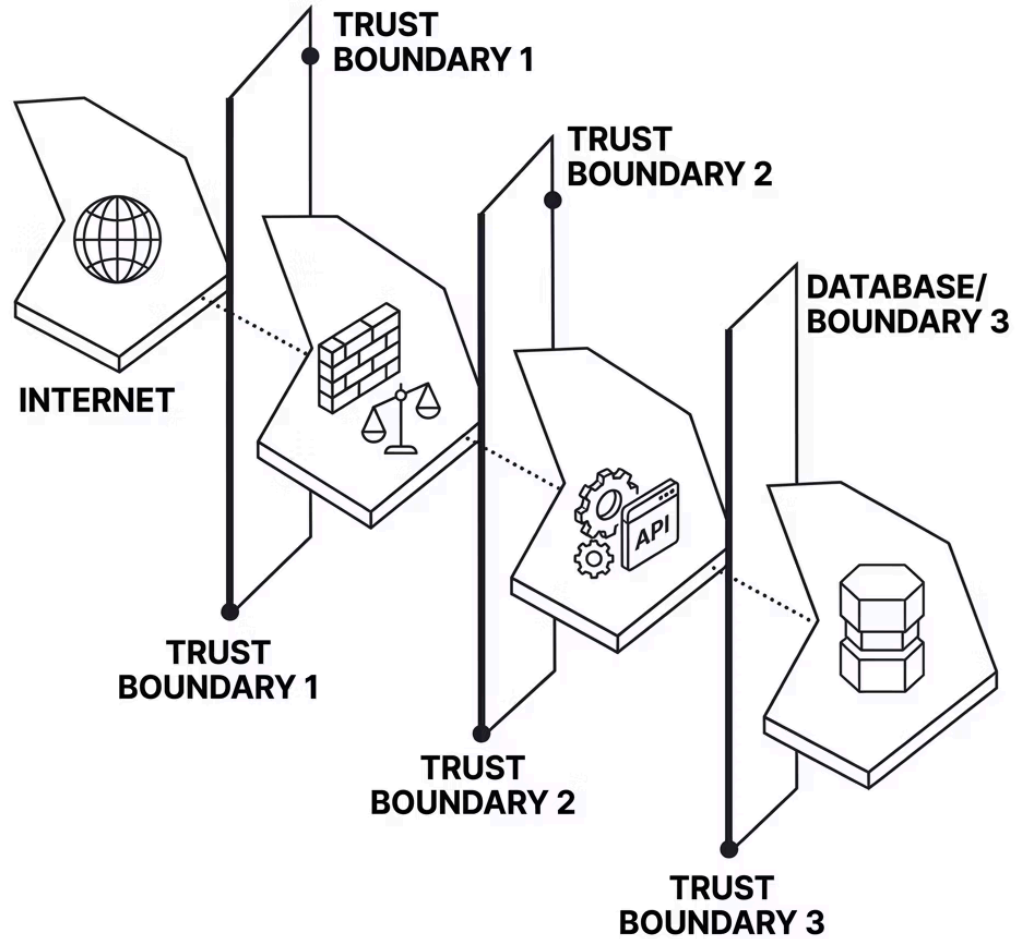
Secure Mortgage Document Exchange

Borrower uploads `income_statement.pdf` — every step is a security control.



- ☐ Encrypt in Transit
- ☐ AuthN + AuthZ
- ☐ File Validation
- ☐ Encrypt at Rest
- ☐ Audit Logging

Understanding Trust Boundaries



At Every Boundary, Ask:

1. Who can cross this boundary?
2. How are they authenticated?
3. What traffic is allowed?
4. What data crosses?
5. Is communication encrypted?
6. How is activity monitored?

☐ Every architectural boundary is also a **security enforcement point**.

Troubleshooting the Request Lifecycle

When an API request fails — troubleshoot systematically. Avoid assuming every failure is a code defect.

01

DNS

Does the hostname resolve?

03

Port

Is the expected port open and listening?

05

TLS

Does certificate validation succeed?

07

Application

Does the request appear in logs?

02

Routing

Can traffic reach the destination network?

04

TCP

Can the connection be established?

06

HTTP

What status code is returned?

08

Downstream

Can the API reach its database and services?

Mini Scenario: Mortgage Document Upload Failure

Symptoms

Borrower uploads `income-proof.pdf` → **Document Upload Failed**

Investigation Step 1 — DNS

```
nslookup api.mortgage.local  
→ NXDOMAIN
```

DNS record missing. Record added. Request still fails.

Investigation Step 2 — Connectivity

```
Port 443 → Connection fails
```

Firewall rule blocking Portal Network → Document API :443. After approved change:

DNS ✓

Routing ✓

TCP ✓

TLS ✓

HTTPS ✓

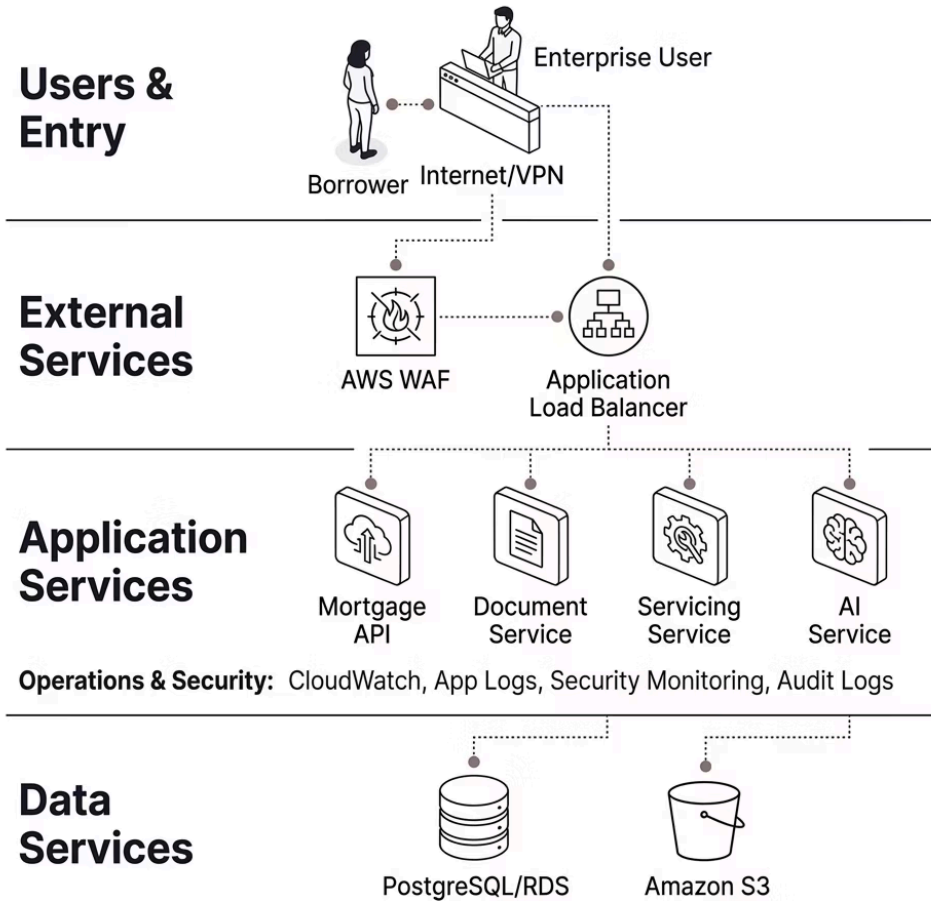
Document API ✓

S3 ✓

Upload Succeeds ✓

 **Key Lesson:** Application failures frequently originate from network infrastructure — not application code.

Mortgage Platform Deployment Topology



Design Objectives

- Minimum public exposure
- Network segmentation by zone
- Encrypted communication everywhere
- Least privilege at every layer
- Continuous monitoring & audit
- Defense in Depth throughout

❏ No backend service is directly reachable from the internet — all traffic enters through WAF and ALB.

Session Summary & Hands-On Labs

You Should Now Understand

- TCP/IP · IP addresses · Ports · DNS · HTTP/HTTPS · TLS
- Routing · NAT · Load Balancing · Cloud Architecture · AWS / VPC
- Zero Trust · Defense in Depth · Security Zones · Firewall · IDS/IPS · WAF · VPN · DMZ
- Security Monitoring · Trust Boundaries · Troubleshooting

Next: Hands-On Labs

- Trace Browser → API → Database → S3
- Inspect HTTP, TLS, and network connections
- Simulate wrong ports and failed connectivity
- Troubleshoot DNS and firewall failures
- Identify trust boundaries
- Draft the mortgage platform deployment topology

📄 **Final Mental Model:** User → DNS → TCP → TLS → WAF → Load Balancer → AuthN/AuthZ → API → Database / Storage → Monitoring